Quantum Information, Dirac Notation and the 16-Qubit Quantum Random Number Generator

A complete, from-the-basics study guide with mathematics, worked examples, circuit diagrams, line-by-line Python (Qiskit) code and viva questions

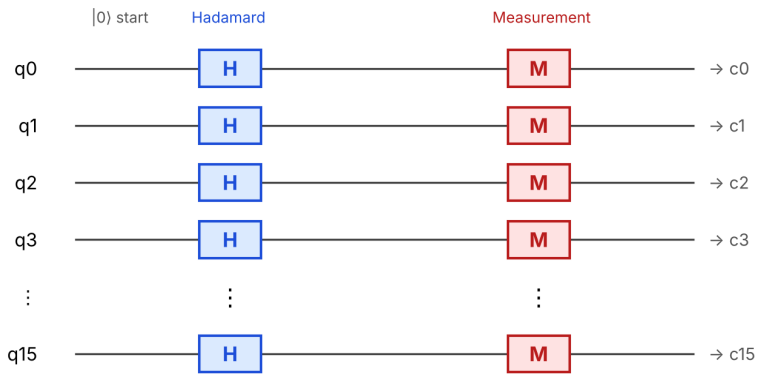


Figure 1: The 16-qubit QRNG circuit: every qubit gets a Hadamard gate (H), then is measured (M).

Python • Qiskit • Qiskit Aer • Linear algebra • Quantum measurement

Contents

1. What is information? What is quantum information?

2. Why do we write |ψ⟩? Dirac (bra-ket) notation

3. Mathematical toolkit (complex numbers, vectors, matrices, tensor product)

4. The qubit in depth: amplitudes, phase and the Bloch sphere

5. Quantum gates: X, Z, H and why H is special

6. Quantum measurement and the Born rule

7. Multi-qubit systems and the Hadamard transform H⊗16

8. Randomness: PRNG, TRNG and QRNG

9. The QRNG circuit and algorithm

10. Libraries: Qiskit and Qiskit Aer

11. Code explained line by line

12. Complete working code (with real output)

13. Extended code: shots, statistics, ranges, bytes, tests

14. Shots, entropy and bit-order (endianness)

15. Using real quantum hardware

16. Common errors and troubleshooting

17. Applications

18. History: who contributed what

19. Viva questions and answers

20. Glossary

21. Resources and conclusion

1. What is Information? What is Quantum Information?

1.1 Information

Information is anything that lets you distinguish between possibilities. If I tell you a coin landed on *heads*, I removed your uncertainty between two possibilities (heads / tails), so I gave you information. The smallest useful amount of information is the answer to one yes/no question. We call it **one bit**.

1.2 Classical information

In an ordinary computer, information is stored in **classical bits**. A bit is always in exactly one definite state, 0 or 1, for example a transistor that is off or on, or a magnetic region that points up or down. Several bits describe bigger things: 3 bits can store one of 8 patterns (000 to 111), and the text, photos and videos on your phone are all long strings of bits.

Two facts about classical information matter for the comparison later: (1) a bit has a definite value even when nobody looks at it, and (2) you can read it any number of times without changing it and you can copy it freely.

1.3 Quantum information

> **Definition**
> **Quantum information** is information stored in the state of a quantum system, such as a photon, an electron spin, a trapped ion or a superconducting circuit, and processed using the rules of quantum mechanics. Its basic unit is the **qubit** (quantum bit).

A qubit has two distinguishable *basis states*, written $|0\rangle$ and $|1\rangle$. Unlike a classical bit, a qubit can also be in a **superposition**, a weighted combination of both basis states. The weights are complex numbers called **probability amplitudes**. The state of one qubit is therefore not “0 or 1” but a pair of numbers (α, β):

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \text{with} \quad |\alpha|^2 + |\beta|^2 = 1$$

1.4 Physical examples of a qubit

Physical system	$\|0\rangle$ stands for	$\|1\rangle$ stands for	Used by
Photon polarisation	Horizontal polarisation	Vertical polarisation	Quantum communication, optical QRNGs
Electron / nuclear spin	Spin up	Spin down	Spin-qubit research, NMR
Trapped ion	Lower energy level	Higher energy level	IonQ, Quantinuum
Superconducting transmon	Ground state of a circuit	First excited state	IBM, Google
Neutral atom	Hyperfine level A	Hyperfine level B	QuEra, Pasqal

1.5 How quantum information works: a worked example

Example: a polarised photon. A polariser acts like a measuring device that asks the photon “are you horizontal or vertical?”. Suppose we prepare a photon in the state

$$|\psi\rangle = (1/\sqrt{2})|H\rangle + (1/\sqrt{2})|V\rangle$$

1. **Before measurement.** The photon is described by two amplitudes, both equal to $1/\sqrt{2} \approx 0.707$. This is a precise, reproducible state. There is nothing vague about it, but it has no single definite horizontal/vertical answer.
2. **Measurement.** We send it to a detector that reports H or V. The probability of each outcome is the *square of the amplitude's magnitude*: $(1/\sqrt{2})^2 = 1/2$. So each outcome has a 50% chance.
3. **Result.** We observe, say, V. This particular outcome could not have been predicted. Quantum theory predicts only the probabilities.
4. **After measurement.** The photon is now in state $|V\rangle$. Measuring again gives V with 100% certainty. The superposition is gone (this is often called *collapse*).
5. **Repeat.** Prepare the identical state 1000 times and measure each. You get about 500 H and 500 V, but the sequence itself is unpredictable. That is exactly what a QRNG exploits.

1.6 The five key properties of quantum information

Property	Meaning	Why it matters here
Superposition	A qubit can be a weighted combination of $\|0\rangle$ and $\|1\rangle$.	The Hadamard gate creates an equal combination, which gives 50/50 outcomes.
Measurement (Born rule)	Measuring gives one classical outcome with probability $\|\text{amplitude}\|^2$.	This is the moment randomness is produced.
Interference	Amplitudes add like waves; they can reinforce or cancel.	Powers quantum algorithms (Grover, Shor). Not needed for QRNG, but H uses it: H·H returns to $\|0\rangle$.
Entanglement	Several qubits can share a joint state that cannot be split into separate states.	Not used in this QRNG, since our 16 qubits are independent.
No-cloning	An unknown quantum state cannot be copied perfectly.	One reason quantum information is useful for security.

1.7 Classical vs quantum information

Feature	Classical bit	Qubit
State	0 or 1	$\alpha\|0\rangle + \beta\|1\rangle$ (α, β complex)
Description of n units	n bits	2^n complex amplitudes
Reading	Does not disturb the value	Measurement changes the state
Result of reading	Deterministic	Probabilistic (Born rule)
Copying	Always possible	Impossible for unknown states (no-cloning)
Operations	Logic gates (AND, OR, NOT)	Unitary gates (reversible)

1.8 A common misconception: how much information can a qubit hold?

Describing n qubits requires 2^n complex amplitudes. For 16 qubits that is 65,536 amplitudes. It is tempting to say a quantum computer "stores" 65,536 values at once. This is misleading. When you measure, you only get **n classical bits** back (Holevo's theorem, 1973). Quantum algorithms are clever precisely because they steer the amplitudes so that the final measurement is useful. For our

QRNG we do not need cleverness: we just want the unpredictable 16 bits that measurement hands us.

2. Why Do We Write |ψ⟩? Dirac (Bra-Ket) Notation

The symbol |ψ⟩ looks strange the first time. It is called a **ket**, part of **Dirac notation** (also called **bra-ket notation**), introduced by the physicist **Paul Dirac** (his notation paper appeared in 1939; his book *The Principles of Quantum Mechanics* in 1930 popularised the approach). The name comes from the word "bracket", split into **bra** ⟨ | and **ket** | ⟩.

2.1 What does the symbol mean?

- **| ⟩ (ket):** a notation that says "the thing written inside is the *label* of a quantum state, and the state is a *column vector*". So |0⟩ means "the state labelled 0", which is the vector $(1, 0)^T$.
- **⟨ | (bra):** the conjugate transpose of the ket, a *row vector*. ⟨0| = (1, 0). For complex entries, you also take the complex conjugate of each entry.
- **⟨a|b⟩ (bracket):** the inner product (a number) between two states. It measures how much the state |b⟩ overlaps with |a⟩.
- **|a⟩⟨b| (outer product):** a matrix. Used to build projectors and gates.

> **Important**
> |0⟩ is **not** the number zero. It is the name of a vector. Writing |0⟩ instead of just 0 tells the reader "this is a quantum state vector", not a number. Likewise |ψ⟩ is "some quantum state called ψ".

2.2 Why did physicists invent this notation?

1. **It separates the state from its coordinates.** A physical state exists independently of how we choose to write it as numbers. The ket |ψ⟩ names the state itself. The column of numbers (α, β) is just its representation in one chosen basis.
2. **It is compact.** The two-qubit state with amplitudes $(0, 0, 1, 0)^T$ is simply |10⟩. For 16 qubits, |1011010010110101⟩ replaces a column of 65,536 numbers almost all zero.
3. **It makes calculations mechanical.** Inner products ⟨a|b⟩, probabilities $|\langle x|\psi\rangle|^2$ and projectors |x⟩⟨x| follow from one consistent grammar.
4. **It works for any quantum system**, even those with infinitely many states (position, momentum), where "column of numbers" is clumsy.
5. **It is the universal language.** Textbooks, Qiskit documentation and research papers all use it.

2.3 Dictionary of common kets

Symbol	Name	Column vector	Meaning
|0⟩	Zero state	$(1, 0)^T$	Definite 0 when measured in the computational basis
|1⟩	One state	$(0, 1)^T$	Definite 1 when measured
|+⟩	Plus state	$(1/\sqrt{2})(1, 1)^T$	(|0⟩+|1⟩)/√2, equal superposition, plus sign
|−⟩	Minus state	$(1/\sqrt{2})(1, -1)^T$	(|0⟩−|1⟩)/√2, equal superposition, minus sign
|ψ⟩	Generic state	$(\alpha, \beta)^T$	Any state; ψ is just a name
|01⟩	Two-qubit state	$(0, 1, 0, 0)^T$	First qubit 0, second qubit 1

2.4 Bra-ket arithmetic with examples

Inner products of basis states. Basis states are *orthonormal*: each has length 1 and different ones are perpendicular.

$$\langle 0|0\rangle = (1,\,0)\cdot(1,\,0)^{\mathsf{T}} = 1, \quad \langle 0|1\rangle = (1,\,0)\cdot(0,\,1)^{\mathsf{T}} = 0, \quad \langle 1|1\rangle = 1$$

Extracting amplitudes. For $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, the bra $\langle 0|$ picks out the amplitude of $|0\rangle$:

$$\langle 0|\psi\rangle = \alpha\langle 0|0\rangle + \beta\langle 0|1\rangle = \alpha$$

Born rule in bra-ket form. The probability of measuring outcome x is

$$P(x) = |\langle x|\psi\rangle|^2$$

Worked example. Let $|\psi\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. Then $\langle 0|\psi\rangle = 1/\sqrt{2}$, so $P(0) = |1/\sqrt{2}|^2 = 1/2$. Likewise $\langle 1|\psi\rangle = 1/\sqrt{2}$ and $P(1) = 1/2$.

Outer product (builds a matrix).

$$|0\rangle\langle 0| = \begin{vmatrix} 1 & 0 \\ 0 & 0 \end{vmatrix} \qquad \text{(projector onto } |0\rangle\text{)}$$

2.5 Why “|” and “⟩” specifically?

The vertical bar and angle bracket are purely a typographic convention. The bar says “a label begins here”, the angle bracket says “label ends, this is a ket”. The paired bra $\langle\psi|$ mirrors it. Putting a bra next to a ket, $\langle a|b\rangle$, closes the angle brackets into a full “bracket”, which is the pun Dirac intended.

2.6 Multi-qubit labels and Qiskit’s ordering

For multiple qubits we write the label as a string of bits: $|q_2 q_1 q_0\rangle$. **Qiskit uses “little-endian” ordering**: qubit 0 is the *right-most* character. So when Qiskit prints 1011010010110101, the right-most 1 belongs to q0 and the left-most 1 belongs to q15. For a QRNG, where all bits are independent and uniform, the order does not affect randomness, but it matters whenever you map bits to specific qubits.

3. Mathematical Toolkit

3.1 Complex numbers

A complex number is $z = a + bi$, where $i^2 = -1$. Its magnitude is $|z| = \sqrt{(a^2 + b^2)}$ and its conjugate is $z^* = a - bi$. Always $|z|^2 = z \cdot z^*$. Amplitudes are complex numbers because quantum states have a *phase*, an angle that matters for interference.

3.2 Vectors and normalisation

A state is a vector whose squared magnitudes sum to 1. This is called **normalisation**, and it guarantees that the probabilities of all outcomes add up to 100%.

Example. Is (0.6, 0.8) a valid qubit state? $0.6^2 + 0.8^2 = 0.36 + 0.64 = 1$. Yes: P(0) = 36%, P(1) = 64%.

3.3 Matrices and matrix multiplication

A quantum gate is a matrix that multiplies the state vector. To multiply a 2×2 matrix by a column vector, take a row of the matrix, multiply it entry-by-entry with the vector, and add:

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix}$$

$$\text{applied to } (x, y)^T \text{ gives } (ax + by, cx + dy)^T$$

3.4 Unitary matrices

A gate must keep the state normalised and be reversible. The matrices that do this are **unitary**: $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose and I the identity. Because the gates are unitary, quantum computation *before measurement* is reversible.

3.5 Tensor product (⊗)

To describe two qubits together, we combine their vectors with the **tensor (Kronecker) product**. For vectors $(a, b)^T$ and $(c, d)^T$:

$$(a, b)^T \otimes (c, d)^T = (ac, ad, bc, bd)^T$$

Example 1. $|0\rangle \otimes |1\rangle = (1, 0) \otimes (0, 1) = (0, 1, 0, 0) = |01\rangle$.

Example 2. $|+\rangle \otimes |+\rangle = (1/\sqrt{2})(1, 1) \otimes (1/\sqrt{2})(1, 1) = ½(1, 1, 1, 1)$. All four basis states $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$ have amplitude ½, so each has probability ¼ = 25%.

The notation $|0\rangle^{\otimes 16}$ means "16 copies of $|0\rangle$ combined by tensor products", which has $2^{16} = 65{,}536$ entries. Most circuit-simulation memory goes into these entries.

4. The Qubit in Depth: Amplitudes, Phase and the Bloch Sphere

4.1 Amplitudes and probabilities

The amplitudes α and β are complex, so each can be written as a magnitude and a phase angle: $\alpha = |\alpha|e^{i\varphi_a}$. Only the magnitudes decide measurement probabilities in the computational basis, but the *relative phase* between α and β decides what happens after further gates. This is why $|+\rangle = (|0\rangle+|1\rangle)/\sqrt{2}$ and $|-\rangle = (|0\rangle-|1\rangle)/\sqrt{2}$ look identical to a computational-basis measurement (both 50/50) yet behave differently under a second Hadamard.

4.2 Bloch sphere

Any single-qubit pure state can be written (up to an unobservable global phase) as

$$|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\varphi}\sin(\theta/2)|1\rangle$$

with angles θ (0 to π) and φ (0 to 2π). These two angles are the latitude and longitude of a point on a sphere of radius 1, the **Bloch sphere** (named after Felix Bloch). The north pole is $|0\rangle$, the south pole is $|1\rangle$, and the equator holds all equal-superposition states such as $|+\rangle$ and $|-\rangle$.

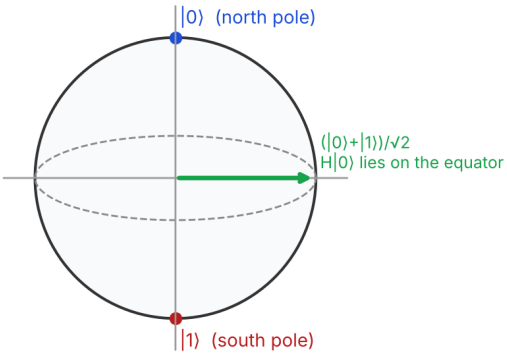


Figure 2: Bloch-sphere picture. The Hadamard gate moves $|0\rangle$ from the north pole to the equator.

Reading the picture for randomness. The probability of measuring 0 is $\cos^2(\theta/2)$. At the north pole (θ = 0) it is 1; at the equator (θ = 90°) it is 1/2. So “maximally random bit” just means “point on the equator”.

4.3 Global vs relative phase

Multiplying the whole state by $e^{i\gamma}$ (global phase) changes nothing observable. Only the relative phase between the $|0\rangle$ and $|1\rangle$ parts is physical.

5. Quantum Gates: X, Z, H and Why H is Special

5.1 Pauli-X (quantum NOT)

$$X = \begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix}$$

$X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$. It flips a bit.

5.2 Pauli-Z (phase flip)

$$Z = \begin{vmatrix} 1 & 0 \\ 0 & -1 \end{vmatrix}$$

$Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$. It changes the relative phase and leaves measurement probabilities of $|0\rangle$ and $|1\rangle$ unchanged.

5.3 Hadamard gate: full derivation

$$H = (1/\sqrt{2}) \times \begin{vmatrix} 1 & 1 \\ 1 & -1 \end{vmatrix}$$

Acting on $|0\rangle$:

$$H|0\rangle = (1/\sqrt{2})(1\cdot 1 + 1\cdot 0,\ 1\cdot 1 + (-1)\cdot 0)^T = (1/\sqrt{2})(1,\ 1)^T = (|0\rangle + |1\rangle)/\sqrt{2} = |+\rangle$$

Acting on $|1\rangle$:

$$H|1\rangle = (1/\sqrt{2})(1\cdot 0 + 1\cdot 1,\ 1\cdot 0 + (-1)\cdot 1)^T = (1/\sqrt{2})(1,\ -1)^T = (|0\rangle - |1\rangle)/\sqrt{2} = |-\rangle$$

Applying H twice returns to the start (interference in action).

$$H|+\rangle = (H|0\rangle + H|1\rangle)/\sqrt{2} = (|+\rangle + |-\rangle)/\sqrt{2} = |0\rangle$$

The $|1\rangle$ terms cancel (destructive interference) and the $|0\rangle$ terms add (constructive interference). In matrix form, $H \cdot H = I$, so H is its own inverse. It is also unitary and Hermitian.

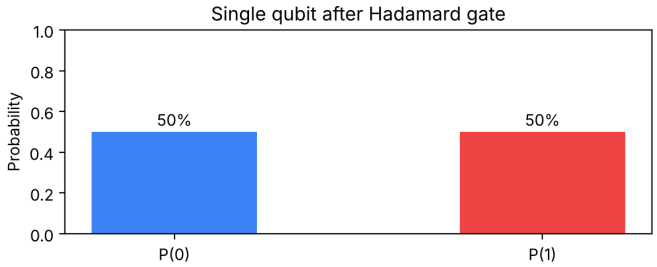


Figure 3: After H on |0⟩, a measurement gives 0 or 1 with equal probability.

5.4 Why the Hadamard gate for a QRNG?

- It maps a definite state $|0\rangle$ to a state with **exactly equal** probabilities 1/2 and 1/2, giving an unbiased bit in the ideal case.
- It needs only one gate per qubit, so it is simple and has the smallest error on real hardware.
- Gates such as X or Z alone cannot create superposition from $|0\rangle$. X gives a certain 1, Z leaves $|0\rangle$ unchanged.

Gate on $\|0\rangle$	Result	P(0)	P(1)	Useful for QRNG?
I (do nothing)	$\|0\rangle$	100%	0%	No, always 0
X	$\|1\rangle$	0%	100%	No, always 1
Z	$\|0\rangle$	100%	0%	No, no effect on probabilities
H	$(\|0\rangle+\|1\rangle)/\sqrt{2}$	50%	50%	Yes, unbiased bit

6. Quantum Measurement and the Born Rule

6.1 The rule

If a system is in state $|\psi\rangle = \Sigma\, c_x|x\rangle$, measuring in the computational basis yields outcome x with probability

$$P(x) = |c_x|^2 = |\langle x|\psi\rangle|^2 \quad \text{(Born rule, Max Born, 1926)}$$

After the result x is obtained, the state becomes $|x\rangle$ (*collapse* or *projection*).

6.2 Worked example: unequal superposition

Take $|\psi\rangle = 0.6|0\rangle + 0.8|1\rangle$. Then $P(0) = 0.36$ and $P(1) = 0.64$. If we used such a state in a QRNG, the bits would be biased towards 1. This is why the *exactly equal* amplitudes from H are important.

6.3 Where does the randomness come from?

According to standard quantum theory, the single outcome of such a measurement is not determined by any hidden pre-existing value. The theory provides only probabilities. Experiments testing Bell inequalities strongly constrain local hidden-variable explanations. This is why a physical QRNG is regarded as a source of *intrinsic* randomness, in contrast to a pseudo-random algorithm that is fully determined by its seed.

> **Important**
> This argument applies to real quantum devices. A simulator on your laptop only *imitates* the probabilities, using a classical pseudo-random generator internally. See Section 8.

7. Multi-Qubit Systems and the Hadamard Transform H⊗16

7.1 Two qubits first

Start with $|00\rangle = |0\rangle \otimes |0\rangle$ and apply H to each qubit. The combined gate is H⊗H:

$$(H|0\rangle) \otimes (H|0\rangle) = \tfrac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

Four outcomes, each with probability $(\tfrac{1}{2})^2 = \tfrac{1}{4}$. Two bits of randomness from two qubits.

7.2 General n qubits

$$H^{\otimes n}|0\rangle^{\otimes n} = (1/\sqrt{(2^n)}) \; \Sigma_{x=0}^{2^n-1} \; |x\rangle$$

For n = 16: every one of the 65,536 bitstrings has amplitude $1/\sqrt{65536} = 1/256$ and probability $(1/256)^2 = 1/65536 \approx 0.001526\%$.

7.3 Independence

The 16 qubits are never connected by two-qubit gates, so the final state is a **product state**: each qubit is its own independent 50/50 coin. The 16-bit result is therefore 16 independent fair coin flips. No entanglement is needed or used.

Number of qubits	Number of outcomes	Amplitudes per outcome	Probability per outcome
1	2	$1/\sqrt{2}$	1/2
2	4	1/2	1/4
8	256	1/16	1/256
16	65,536	1/256	1/65,536
32	4,294,967,296	1/65,536	$\approx 2.3\times10^{-10}$

Memory note: a full statevector simulation of n qubits needs 2^n complex numbers. At 16 qubits that is tiny, but at 50 qubits it is about 1.1×10^{15} numbers, which is why large circuits cannot be simulated on a laptop. (Aer can still run 16-qubit Hadamard-and-measure circuits instantly.)

8. Randomness: PRNG, TRNG and QRNG

Type	How it works	Strength	Weakness
PRNG (pseudo-random)	Deterministic algorithm from a seed (e.g. Mersenne Twister)	Fast, reproducible	Same seed gives same sequence; predictable if state is known
CSPRNG	PRNG designed to resist prediction (e.g. OS /dev/urandom, ChaCha20)	Good for most cryptography	Still seeded from some entropy source
TRNG (classical hardware)	Samples physical noise (thermal noise, jitter)	Non-deterministic	Noise may be biased and may be influenced by environment
QRNG	Measures a quantum process (photon path, vacuum fluctuations, qubit measurement)	Randomness rooted in quantum theory	Needs hardware, calibration, and post-processing

8.1 Is the simulator output really quantum-random?

No. When you run AerSimulator on a laptop, it computes the exact probabilities of the quantum circuit and then draws samples using a classical pseudo-random number generator. The *statistics* are those of an ideal QRNG, but the *source* is classical. This guide is therefore a faithful **demonstration** of the QRNG principle. For physical quantum randomness, the same circuit must run on a quantum processor (Section 15), or you must use a dedicated QRNG device.

8.2 What real QRNGs measure

- **Photon path:** a single photon meets a 50/50 beam splitter and a detector registers which output port it took.
- **Vacuum fluctuations:** measuring quantum noise of the electromagnetic vacuum with a homodyne detector (used in published high-rate QRNGs).
- **Photon arrival time:** timing of detection events from a weak light source.
- **Qubit measurement:** our circuit, executed on a real quantum processor.

9. The QRNG Circuit and Algorithm

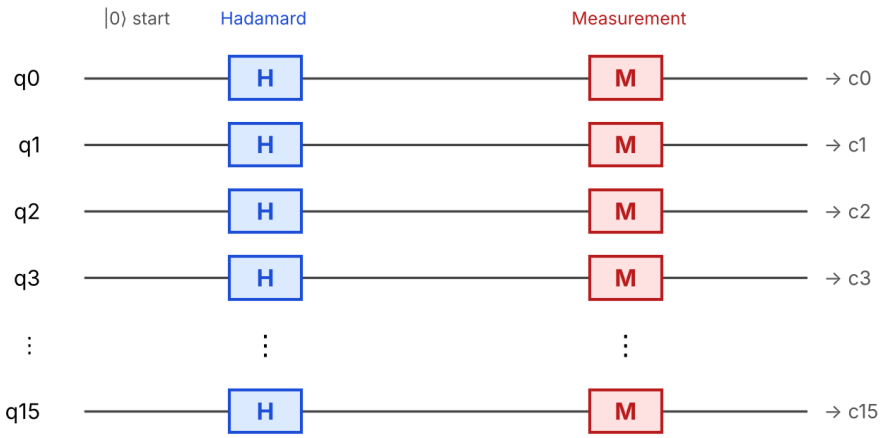


Figure 4: Circuit (q0 ... q15 shown with q4–q14 abbreviated). Each wire starts in |0⟩, receives H, then is measured into classical bit c_i.

9.1 Algorithm

```
START
 1. Create a quantum circuit with 16 qubits and 16 classical bits.
 2. All qubits begin in state |0>.
 3. Apply a Hadamard gate to each qubit      -> equal superposition.
 4. Measure all 16 qubits into the 16 classical bits.
 5. Run the circuit on a backend (simulator or real QPU).
 6. Read the measured 16-character bitstring.
 7. Convert the bitstring to a decimal integer (0 .. 65,535).
 8. Print / use the random number.
STOP
```

9.2 Flow (text flowchart)

```
  [Create 16-qubit circuit]
            |
            v
  [Apply H to every qubit]
            |
            v
   [Measure all qubits]
            |
            v
    [Transpile for backend]
            |
            v
  [Run: shots = 1  (or more)]
            |
            v
   [get_counts() -> bitstring]
            |
            v
  [int(bitstring, 2) -> decimal]
            |
            v
          [Output]
```

9.3 Probability model

$$P(\text{any specific 16-bit string}) = (1/2)^{16} = 1/65{,}536$$

Expected value of the decimal number: (0 + 65,535)/2 = 32,767.5. Over many runs, the average of generated numbers should approach this.

10. Libraries: Qiskit and Qiskit Aer

10.1 Qiskit

Qiskit is an open-source Python SDK for quantum computing, first released by IBM in 2017. It lets you build circuits, apply gates, transpile them for hardware, and run them on simulators or real devices. We use `QuantumCircuit` (to build the circuit) and `transpile` (to adapt it to a backend).

10.2 Qiskit Aer

Qiskit Aer is a high-performance simulator package. It runs circuits on your classical computer and supports several methods: statevector, density matrix, stabilizer, matrix-product-state and others, plus configurable noise models. We use `AerSimulator`.

10.3 Installation

```
pip install qiskit qiskit-aer
pip show qiskit
pip show qiskit-aer
```

Important
Qiskit has changed significantly across versions (for example, the old `execute()` and `Aer.get_backend()` style from before Qiskit 1.0 was removed). The code in this guide uses the current style: `simulator.run(transpile(...))`. I ran it successfully with a recent Qiskit and qiskit-aer installation. If an import fails, check your installed versions.

11. Code Explained Line by Line

Step 1. Imports

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
```

- `from X import Y` loads the name Y from package X.
- `QuantumCircuit` is the class we use to describe a circuit.
- `transpile` rewrites a circuit into gates a particular backend understands.
- `AerSimulator` is the simulator backend.

Step 2. Create the circuit

```
qc = QuantumCircuit(16, 16)
```

The signature is `QuantumCircuit(num_qubits, num_classical_bits)`. We get 16 qubits (all |0⟩) and 16 classical bits (all 0). Classical bits are needed because measurement results are classical information and must be stored somewhere.

Step 3. Hadamard on every qubit

```
for i in range(16):
    qc.h(i)
```

`range(16)` yields 0, 1, ..., 15 (16 numbers; 16 itself is excluded). `qc.h(i)` appends an H gate on qubit i. Equivalent shortcut: `qc.h(range(16))`.

Step 4. Measurement

```
qc.measure(range(16), range(16))
```

Signature: `measure(qubits, clbits)`. Qubit i is measured and its result is stored in classical bit i. Without this line, the circuit would have no outputs to read.

Qubit	→	Classical bit
q0	→	c0
q1	→	c1
...	...	...
q15	→	c15

Step 5. Draw the circuit

```
print(qc.draw())
```

`draw()` returns an ASCII drawing; `print` shows it. You may use `qc.draw('mpl')` for a Matplotlib figure.

Step 6. Create the simulator

```
simulator = AerSimulator()
```

Creates a backend object. By default it chooses a suitable simulation method automatically.

Step 7. Transpile

```
compiled_circuit = transpile(qc, simulator)
```

Transpilation is to quantum circuits what compilation is to source code: the circuit is decomposed into the backend's native gates and optimised. For this tiny circuit nothing dramatic changes, but the step is mandatory in good practice and essential for real hardware.

Step 8. Run

```
result = simulator.run(compiled_circuit, shots=1).result()
```

- `run(circuit, shots=1)` submits a job. One shot = one full execution and measurement = one 16-bit sample.
- `.result()` blocks until the job finishes and returns the result object.

Step 9. Counts

```
counts = result.get_counts()
```

Returns a dictionary {bitstring: how-many-times}. With one shot: {'1001000100101110': 1}.

Step 10. Extract the bitstring

```
random_binary = list(counts.keys())[0]
```

`counts.keys()` gives the keys, `list(...)` turns them into a list, and `[0]` picks the first (and only) one.

Step 11. Binary to decimal

```
random_decimal = int(random_binary, 2)
```

`int(text, base)` parses text as an integer in the given base. Example with our run: $1001000100101110_2 = 2^{15} + 2^{12} + 2^8 + 2^5 + 2^3 + 2^2 + 2^1 = 32768 + 4096 + 256 + 32 + 8 + 4 + 2 =$ **37166**.

Step 12. Print

```
print("Random 16-bit Binary Number:", random_binary)
print("Random Decimal Number:", random_decimal)
```

12. Complete Working Code (with Real Output)

The listing in your earlier notes was cut off at step 10 and its line breaks were lost. Here is the complete, correctly formatted program. I ran it to confirm that it works.

```
# qrng_16.py : 16-qubit Quantum Random Number Generator
# Requires:  pip install qiskit qiskit-aer

# 1. Import required libraries
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

# 2. Create a circuit with 16 qubits and 16 classical bits
qc = QuantumCircuit(16, 16)

# 3. Put every qubit into equal superposition with a Hadamard gate
for i in range(16):
    qc.h(i)

# 4. Measure all qubits into the classical bits
qc.measure(range(16), range(16))

# 5. Display the circuit
print("Quantum Circuit:")
print(qc.draw())

# 6. Create the simulator backend
simulator = AerSimulator()

# 7. Transpile for the backend
compiled_circuit = transpile(qc, simulator)

# 8. Execute: one shot = one 16-bit sample
result = simulator.run(compiled_circuit, shots=1).result()

# 9. Get the measurement counts, e.g. {'1001000100101110': 1}
counts = result.get_counts()

# 10. Extract the 16-bit binary string
random_binary = list(counts.keys())[0]

# 11. Convert binary to decimal (0 .. 65535)
random_decimal = int(random_binary, 2)

# 12. Show the results
print("Random 16-bit Binary Number:", random_binary)
print("Random Decimal Number:", random_decimal)
```

Sample output from one actual run

```
Random 16-bit Binary Number: 1001000100101110
Random Decimal Number: 37166
```

Your numbers will differ every time you run it. That is the expected behaviour.

13. Extended Code: Shots, Statistics, Ranges, Bytes, Tests

13.1 Many 16-bit numbers in one run

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

qc = QuantumCircuit(16, 16)
qc.h(range(16))
qc.measure(range(16), range(16))

sim = AerSimulator()
tqc = transpile(qc, sim)

# memory=True returns the shot-by-shot list in order (not just totals)
result = sim.run(tqc, shots=10, memory=True).result()
samples = result.get_memory()          # list of 10 bitstrings
numbers = [int(s, 2) for s in samples]
for s, n in zip(samples, numbers):
    print(s, n)
```

Why `memory=True`? `get_counts()` collapses identical results into a dictionary and loses the order. For a random *sequence* you want the per-shot list.

13.2 A reusable function

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

_sim = AerSimulator()

def quantum_random_bits(n_bits: int) -> str:
    """Return a string of n_bits random bits using one n_bits-qubit circuit."""
    qc = QuantumCircuit(n_bits, n_bits)
    qc.h(range(n_bits))
    qc.measure(range(n_bits), range(n_bits))
    tqc = transpile(qc, _sim)
    res = _sim.run(tqc, shots=1, memory=True).result()
    return res.get_memory()[0]

def quantum_random_int(n_bits: int = 16) -> int:
    return int(quantum_random_bits(n_bits), 2)

print(quantum_random_int(16))     # 0 .. 65535
print(quantum_random_int(8))      # 0 .. 255
```

13.3 Random number in a custom range (rejection sampling)

Suppose you want a number from 1 to 100. A tempting shortcut is `x % 100 + 1`, but this makes some values slightly more likely whenever 65,536 is not divisible by 100 (a *modulo bias*). The unbiased method is **rejection sampling**: throw away values that fall in the uneven leftover zone and draw again.

```
def quantum_randrange(low: int, high: int) -> int:
    """Unbiased integer in [low, high] inclusive, using 16-bit draws."""
    span = high - low + 1
    if span > 65536:
        raise ValueError("range too large for 16 bits; use more qubits")
    limit = (65536 // span) * span         # largest multiple of span <= 65536
    while True:
        x = quantum_random_int(16)
        if x < limit:                      # accept
```

```
            return low + (x % span)
        # else: reject and draw again

print(quantum_randrange(1, 100))
```

Why it works: values 0 .. limit−1 split into equal groups of size *span*, so each remainder 0 .. span−1 appears equally often.

13.4 Bytes and hexadecimal

```
def quantum_random_bytes(n: int) -> bytes:
    bits = quantum_random_bits(8 * n)      # note: needs 8n qubits
    return int(bits, 2).to_bytes(n, "big")

print(quantum_random_bytes(2).hex())       # e.g. '912e'
```

13.5 Statistical check of the output

We can check that each bit is about 50% ones. This is a *sanity check* only. A proper QRNG evaluation uses test suites such as NIST SP 800-22 or Dieharder.

```
from collections import Counter
from math import sqrt

shots = 2000
res = sim.run(tqc, shots=shots, memory=True).result()
samples = res.get_memory()

# fraction of ones at each bit position (string index 0 = c15, last = c0)
ones = [0] * 16
for s in samples:
    for pos, ch in enumerate(s):
        if ch == "1":
            ones[pos] += 1

for pos, k in enumerate(ones):
    p = k / shots
    z = (k - shots / 2) / sqrt(shots * 0.25)     # standard score
    print(f"bit position {pos:2d}: P(1) = {p:.3f}   z = {z:+.2f}")
```

Reading the result. For a fair coin, the z-score should usually lie between −2 and +2 (about 95% of the time). An occasional |z| slightly above 2 in one of 16 positions is normal. Consistently large |z| would suggest bias. My test run with 1000 shots gave per-bit P(1) values between 0.471 and 0.510, all consistent with 0.5. It also gave 990 distinct bitstrings among 1000 draws, which agrees with expectation: with 65,536 possible values, a few repeats appear (the birthday effect), but nearly all draws are different.

13.6 Draw the histogram of bits

```
import matplotlib.pyplot as plt
plt.bar(range(15, -1, -1), [k / shots for k in ones])
plt.axhline(0.5, color="red", linestyle="--")
plt.xlabel("Qubit index"); plt.ylabel("Fraction of 1s")
plt.title("Per-qubit frequency of 1")
plt.show()
```

14. Shots, Entropy and Bit-Order

14.1 Shots

Setting	What you get	Typical use
shots=1	One 16-bit sample	One random number
shots=1000, get_counts()	Dictionary of bitstrings → counts	Studying a distribution
shots=1000, memory=True	Ordered list of 1000 samples	Random sequence / stream

Important
With 16 qubits and 1000 shots, you will see up to 1000 different bitstrings, **not** all 65,536. Most possible outcomes simply never occur in so few samples.

14.2 Shannon entropy

$$H(X) = -\Sigma\, p_i \log_2 p_i$$

For uniform $p_i = 1/65{,}536$ over 65,536 outcomes: $H = -65{,}536 \times (1/65{,}536) \times \log_2(1/65{,}536) = \log_2(65{,}536) =$ **16 bits**. Each bit contributes 1 bit of entropy: 16 independent fair bits give 16 bits.

Single biased bit example. If $P(1) = 0.6$, then $H = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.971$ bits. Slight bias already loses entropy. Real QRNG hardware therefore applies a **randomness extractor** (e.g. Toeplitz hashing) to remove residual bias.

14.3 Bit order (endianness)

Qiskit prints the classical register with c15 on the left and c0 on the right. In `int(s, 2)`, the left character is the most significant bit. So c15 is the highest-value bit (2^{15}) and c0 is the lowest (2^0). Since every bit is an independent fair coin, any consistent ordering gives a uniform number.

15. Using Real Quantum Hardware

To get randomness from an actual quantum processor, the same circuit is sent to a cloud device. IBM provides access through **IBM Quantum Platform** and the `qiskit-ibm-runtime` package. The following sketch shows the idea. Details such as authentication and class names change between releases, so consult the current IBM Quantum documentation before running it.

```
# Illustrative sketch. Check IBM Quantum docs for the current API.
# pip install qiskit-ibm-runtime
from qiskit import QuantumCircuit, transpile
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler

service = QiskitRuntimeService()                # needs a saved IBM Quantum account/API key
backend = service.least_busy(operational=True, simulator=False)

qc = QuantumCircuit(16, 16)
qc.h(range(16))
qc.measure(range(16), range(16))

tqc = transpile(qc, backend)                    # maps to the device's native gates
sampler = Sampler(mode=backend)
job = sampler.run([tqc], shots=1)
counts = job.result()[0].data.c.get_counts()    # register is named 'c' by default
print(counts)
```

Important
Real devices are noisy. Gate errors and readout errors can bias bits. A real-hardware QRNG should be tested for bias and post-processed with an extractor before cryptographic use. Also, quantum cloud runs have queue times and usage limits.

16. Common Errors and Troubleshooting

Problem	Cause	Fix
ModuleNotFoundError: qiskit_aer	Aer not installed	pip install qiskit-aer
ImportError: cannot import name 'execute' / 'Aer'	Old-style code from pre-1.0 tutorials	Use simulator.run(transpile(qc, simulator))
IndexError on list(counts.keys())[0]	No measurement in circuit, so counts are empty	Add qc.measure(...)
Output shorter than 16 bits	Fewer classical bits than expected, or you printed a different register	Use QuantumCircuit(16, 16) and measure all
Same number each run	Seed fixed: seed_simulator=...	Remove the seed argument
Tutorial shows q15 on top, you expect q0	Different drawing / ordering conventions	Remember: Qiskit strings are little-endian (q0 right-most)
Pip permission errors	System-wide install not allowed	Use a virtual environment: python -m venv venv

17. Applications of Quantum Random Numbers

Area	Use	Note
Cryptography	Key generation, nonces, salts	Requires validated hardware and extraction
Quantum key distribution	Random basis and bit choice (e.g. BB84)	Randomness quality is critical to security
Scientific simulation	Monte Carlo methods	PRNGs are usually sufficient; quantum randomness is rarely required
Gaming / lotteries	Auditable fair outcomes	Regulatory approval needed
Authentication	Tokens, challenge values	Combine with CSPRNG in practice
Fundamental physics	Bell-test experiments	Random setting choices

Be honest about limits: for everyday software, a well-seeded cryptographically secure PRNG (operating-system randomness) is already adequate. QRNGs matter most where assurance of physical unpredictability is required.

18. History: Who Contributed What

Who	When	Contribution
Max Planck	1900	Quantum hypothesis: energy comes in discrete packets
Albert Einstein	1905	Light quanta and the photoelectric effect
Niels Bohr	1913	Quantised atomic model
Max Born	1926	Probability interpretation of the wavefunction (Born rule)
Paul Dirac	1930 / 1939	*Principles of Quantum Mechanics* and the bra-ket notation
John von Neumann	1932	Mathematical foundations of quantum mechanics and measurement
Felix Bloch	1946	Bloch-vector description (the Bloch sphere is named after him)
Alexander Holevo	1973	Bound on classical information extractable from qubits
Wootters, Zurek; Dieks	1982	No-cloning theorem
Richard Feynman	1981–82	Proposed simulating quantum physics with quantum computers
David Deutsch	1985	Universal quantum computer model
Peter Shor / Lov Grover	1994 / 1996	Factoring algorithm / quantum search
Benjamin Schumacher	1995	Coined the term “qubit” in a published paper
IBM Qiskit team	2017	Open-source Qiskit framework released

Jacques Hadamard (1865–1963) was a French mathematician. The Hadamard *matrix* is named after his work on determinants (1893); he did not work on quantum computing. The matrix was adopted into quantum computing later.

Who invented the QRNG? There is no single inventor. Optical and photon-counting random generators were explored from the 1980s onward, and commercial devices appeared in the 2000s. The circuit in this guide is a modern educational realisation of the same principle.

19. Viva Questions and Answers

Q1. What is a qubit?

The basic unit of quantum information; a two-level quantum system whose state is $\alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$.

Q2. What does $|\psi\rangle$ mean and why is it written like that?

It is a ket in Dirac notation: the name of a quantum state, represented as a column vector. The notation separates the abstract state from its coordinates and makes inner products and probabilities easy to write.

Q3. What is a bra?

$\langle\psi|$ is the conjugate transpose of $|\psi\rangle$ (a row vector). $\langle a|b\rangle$ is the inner product.

Q4. Is $|0\rangle$ the same as the number 0?

No. $|0\rangle$ is the vector $(1, 0)^T$, the basis state labelled 0. When measured it gives the bit 0 with certainty.

Q5. What is superposition?

A quantum state that is a linear combination of basis states, with complex amplitudes.

Q6. State the Born rule.

$P(x) = |\langle x|\psi\rangle|^2$: the probability of an outcome is the squared magnitude of its amplitude.

Q7. What is the Hadamard gate's matrix?

$H = (1/\sqrt{2})[[1, 1], [1, -1]]$. It maps $|0\rangle$ to $(|0\rangle+|1\rangle)/\sqrt{2}$ and $|1\rangle$ to $(|0\rangle-|1\rangle)/\sqrt{2}$.

Q8. What is H·H?

The identity matrix, so applying H twice returns the original state.

Q9. Why 16 qubits?

To get a 16-bit result with $2^{16} = 65{,}536$ equally likely outcomes (0 to 65,535).

Q10. Are the 16 qubits entangled in this circuit?

No. There are no two-qubit gates, so the state is a product of 16 independent single-qubit states.

Q11. What is the probability of any given 16-bit output?

$(1/2)^{16} = 1/65{,}536 \approx 0.0015\%$.

Q12. What is a classical register and why is it needed?

A set of classical bits that stores measurement outcomes.

Q13. What is transpilation?

Rewriting a circuit into the native gates and connectivity of a target backend, with optimisation.

Q14. What is a shot?

One execution of the circuit followed by measurement of all measured qubits.

Q15. What is the difference between get_counts() and get_memory()?

get_counts() summarises how many times each bitstring occurred; get_memory() returns the ordered list of per-shot results.

Q16. Does the AerSimulator provide true randomness?

No. It reproduces quantum probabilities but draws samples with a classical pseudo-random generator. True physical randomness needs real quantum hardware or a QRNG device.

Q17. Why is modulo reduction a bad way to get a number in 1..100?

It introduces a slight bias when 65,536 is not divisible by 100. Use rejection sampling.

Q18. What is the Shannon entropy of 16 ideal random bits?

16 bits (maximum).

Q19. What is the no-cloning theorem?

An unknown quantum state cannot be copied exactly.

Q20. Why can’t a qubit store 2^n classical numbers?

Although n qubits are described by 2^n amplitudes, measurement yields only n classical bits (Holevo bound).

Q21. Qiskit prints 0001. Which qubit is 1?

Qubit 0. Qiskit strings are little-endian: the right-most character is q0.

Q22. Can this run on a real quantum computer?

Yes, using a hardware backend (for example through IBM Quantum with qiskit-ibm-runtime), but noise and bias must be handled.

20. Glossary

Term	Definition
Amplitude	Complex number multiplying a basis state; its squared magnitude is a probability
Backend	Device or simulator that executes circuits
Basis state	One of the reference states, e.g. $\|0\rangle$ and $\|1\rangle$
Bell inequality	A constraint satisfied by local hidden-variable theories; quantum experiments violate it
Bloch sphere	Geometric picture of a single qubit state
Bra / Ket	$\langle\psi\|$ row vector / $\|\psi\rangle$ column vector
Classical register	Bits that hold measurement results
Collapse	Update of the state to the observed outcome after measurement
Computational basis	The basis $\{\|0\rangle, \|1\rangle\}$ (and its products for several qubits)
Endianness	Convention for which end of a bitstring is the least significant bit
Entanglement	Correlation between qubits not describable by separate states
Entropy	Measure of unpredictability, in bits
Hermitian	A matrix equal to its conjugate transpose
Inner product	$\langle a\|b\rangle$; a number measuring overlap
Normalisation	Requirement that probabilities sum to 1
Orthonormal	Perpendicular and of length 1
Phase	Angle of a complex amplitude
Product state	Joint state that factors into independent single-qubit states
QPU	Quantum processing unit
Qubit	Quantum bit
Shot	One run and measurement of a circuit
Statevector	Vector of all 2^n amplitudes of an n-qubit state
Tensor product	Operation that joins separate systems
Transpiler	Tool that rewrites circuits for a backend
Unitary	Matrix with $U^\dagger U = I$; represents a reversible quantum operation

21. Resources and Conclusion

Resources

- IBM Quantum Learning and documentation: quantum.cloud.ibm.com/docs
- Qiskit Aer documentation: qiskit.github.io/qiskit-aer
- Nielsen & Chuang, *Quantum Computation and Quantum Information* (the standard textbook)
- NIST SP 800-22 (statistical tests for random number generators)

Conclusion

A 16-qubit QRNG applies an **H** gate to 16 qubits that start in $|0\rangle$, producing the equal superposition $H^{\otimes 16}|0\rangle^{\otimes 16}$, in which each of 65,536 bitstrings has probability 1/65,536. Measuring all qubits yields 16 independent fair bits, which we convert to a decimal number with `int(bits, 2)`. The Dirac ket $|\psi\rangle$ is simply the compact name for the state vector and makes this entire calculation easy to write. Remember the honest caveat: the simulator reproduces the mathematics, whereas real quantum randomness requires physical quantum hardware.

Five topics to focus on for your practical exam

1. Qubit, ket notation and computational basis
2. Superposition, Born rule and measurement
3. Hadamard matrix and the derivation $H|0\rangle = |+\rangle$
4. QuantumCircuit, AerSimulator, transpile, run, get_counts
5. Shots, bit-order, and why a simulator is not a true QRNG